

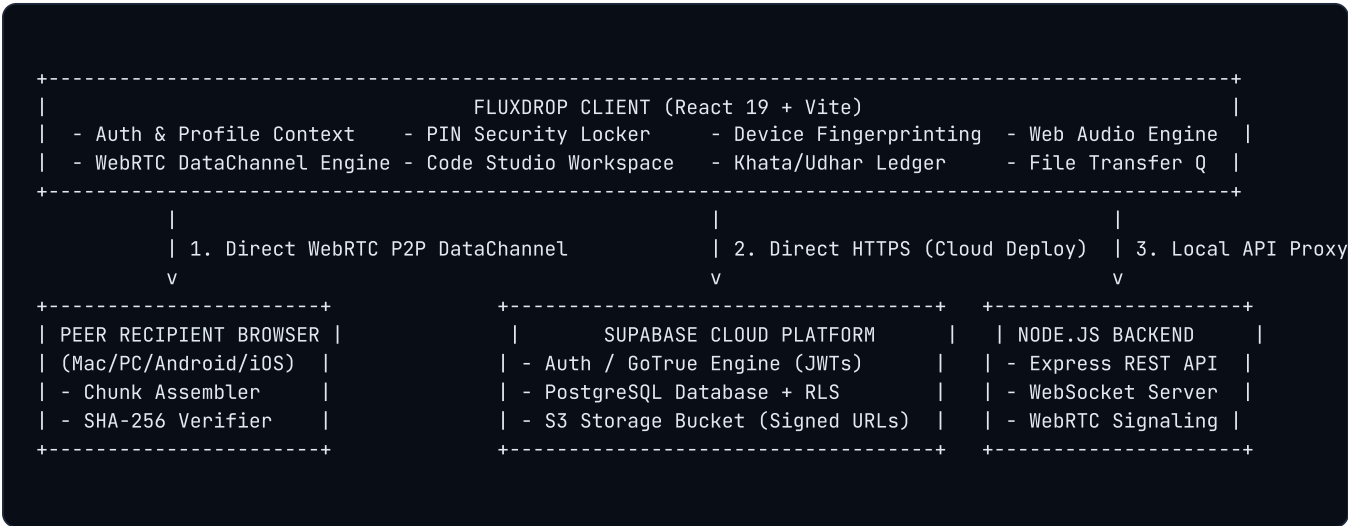
Technical Requirement Document (TRD)

FluxDrop — Architecture, Data Schemas, Cryptographic Protocols & System Interfaces

Document Version	v1.2.0 (Production Architecture)	Target Systems	Web, Node.js Backend, Supabase BaaS, Vercel CDN
Lead Architect	FluxDrop Engineering Team	Live Deployment	https://fluxdrop-psi.vercel.app/
Primary Tech Stack	React 19, TypeScript, Node.js v24, Express, WebSocket, WebRTC, Supabase (PostgreSQL, GoTrue, S3)	Database Project Ref	jyuazhyurgmjshbhvulp

1. System Architecture Overview

FluxDrop implements a resilient **Hybrid Edge-Cloud Architecture**. High-volume file payloads are transmitted directly between client browsers via WebRTC SCTP DataChannels. When peers are not co-located or are behind restrictive symmetric NATs, payloads automatically route through Supabase Storage S3 buckets. The signaling and state sync layer is handled through a dual-mode communication pipeline: local Node.js Express/WebSocket daemon during local development, and direct Supabase GoTrue + PostgREST integration on cloud edge deployments (Vercel).



2. Technology Stack & Component Architecture

Client-Side Framework

- **Core:** React 19.x with TypeScript 5.7
- **Build Engine:** Vite 8.3 with Rolldown tree-shaking
- **Styling:** Tailwind CSS & CSS Glassmorphism tokens
- **Motion:** Framer Motion & Canvas Confetti
- **Icons:** Lucide React & Custom SVG Logos
- **P2P:** WebRTC RTCPeerConnection (SCTP DataChannel)

Backend & Signaling

- **Runtime:** Node.js v24.15.0 LTS
- **Server Framework:** Express 4.21 with TypeScript
- **WebSocket Engine:** ws library v8.18 for signaling
- **File Buffer:** Multer 1.4 for multipart file relays
- **Security:** bcryptjs, jsonwebtoken, CORS whitelist
- **Process Runner:** tsx for hot reload & execution

Cloud Persistence & Storage (Supabase)

- **Database:** PostgreSQL with Row-Level Security (RLS)
- **Authentication:** Supabase GoTrue with secure session tokens
- **Object Storage:** S3-compliant bucket fluxdrop-files
- **Client SDK:** @supabase/supabase-js v2.117
- **Security Policies:** Permissive public client rules on all entities

Cloud Infrastructure & CI/CD

- **Frontend Hosting:** Vercel Global Edge Network
- **Routing:** Single Page Application (SPA) rewrite rules
- **Source Control:** Git + GitHub (Branch: main)
- **Zero Mixed-Content:** 100% TLS/HTTPS communication

3. Supabase Relational Database Schema

The system utilizes a normalized relational schema within the `public` schema of PostgreSQL, mapped to Supabase Auth UUIDs.

3.1 Table: `public.users`

Column	Type	Description & Constraints
<code>id</code>	UUID PRIMARY KEY	Maps directly to <code>auth.users.id</code> from GoTrue.
<code>username</code>	TEXT UNIQUE NOT NULL	Public display handle (alphanumeric, 3-30 chars).
<code>email</code>	TEXT UNIQUE NOT NULL	Normalized lowercase email identifier.
<code>password_hash</code>	TEXT NOT NULL	Bcrypt hash for local auth or <code>managed_by_supabase_auth</code> .
<code>storage_used</code>	BIGINT DEFAULT 0	Current consumed cloud buffer storage in bytes.
<code>storage_limit</code>	BIGINT DEFAULT 10737418240	Maximum allocated cloud storage cap (10GB default).
<code>security_question</code>	TEXT	Selected challenge question for self-service PIN recovery.
<code>security_answer_hash</code>	TEXT	SHA-256 hash of normalized lowercase answer string.

3.2 Table: `public.pins`

Column	Type	Description & Constraints
<code>id</code>	UUID PRIMARY KEY	Default <code>gen_random_uuid()</code> .
<code>user_id</code>	UUID UNIQUE NOT NULL	Foreign Key → <code>public.users(id)</code> ON DELETE CASCADE.
<code>pin_hash</code>	TEXT NOT NULL	SHA-256 or bcrypt hash of 4-6 digit workspace PIN.
<code>failed_attempts</code>	INTEGER DEFAULT 0	Consecutive failed verification counter.
<code>locked_until</code>	TIMESTAMPTZ	Brute-force penalty timestamp (exponential backoff).

3.3 Tables: public.devices , public.files , public.udhar_*

Table Name	Key Attributes	Primary Purpose
public.devices	id, user_id, device_name, device_type, platform, browser, is_online, last_seen	Tracks authenticated device perimeter, heartbeats, and pairing trust.
public.files	id, owner_id, storage_key, original_name, mime_type, size, checksum, category	Metadata registry for cloud-buffered and Code Studio shared files.
public.udhar_customers	id, user_id, name, phone, notes, total_gave, total_got, balance	Directory of counterparties in Khata financial tracking.
public.udhar_transactions	id, user_id, customer_id, type ('gave' 'got'), amount, payment_mode, is_settled	Audit log of individual credit and payment settlements.

4. Cryptographic Security & Recovery Protocol

4.1 Client-Side SHA-256 Hashing Implementation

To prevent sensitive cleartext passwords or recovery answers from ever hitting transit unencrypted, the frontend employs native Web Cryptography API primitives:

```
async function hashSha256(text: string): Promise<string> {
  const msgUint8 = new TextEncoder().encode(text.trim().toLowerCase());
  const hashBuffer = await crypto.subtle.digest('SHA-256', msgUint8);
  const hashArray = Array.from(new Uint8Array(hashBuffer));
  return hashArray.map((b) => b.toString(16).padStart(2, '0')).join('');
}
```

4.2 Dual-Mode API Routing Architecture

The client API layer in `frontend/src/services/api.ts` dynamically adjusts its communication protocol depending on host execution context:

Local Development Mode (localhost)

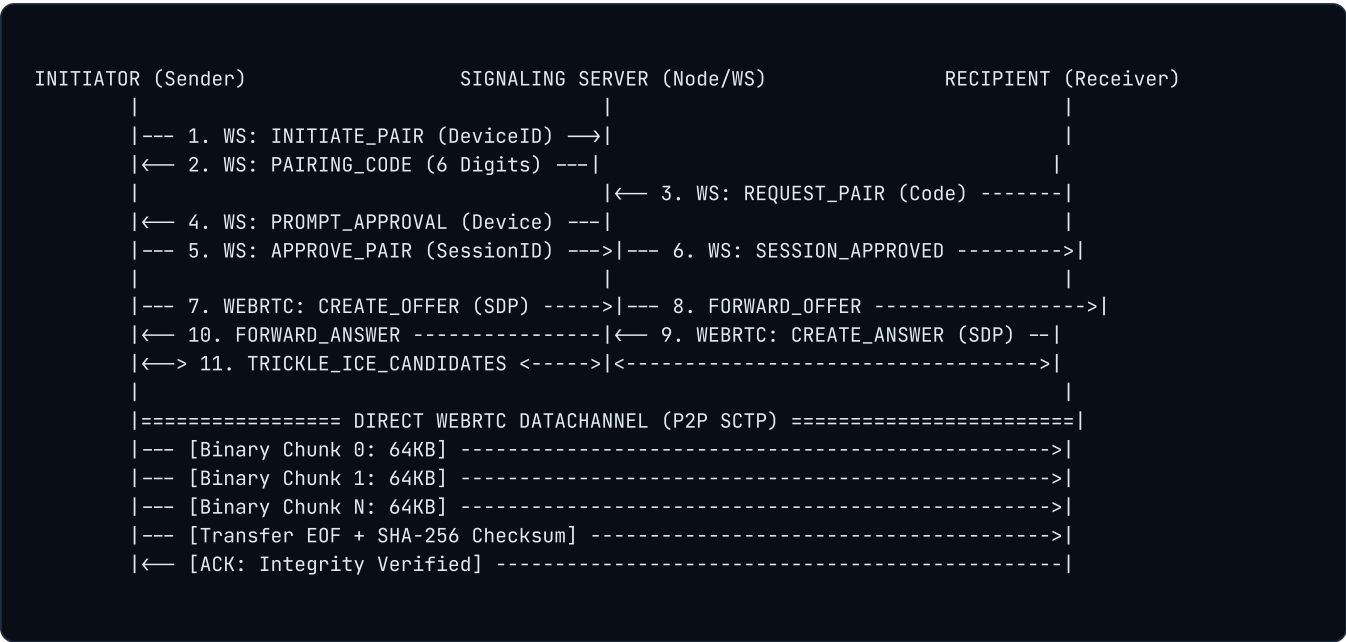
Routes requests to Express backend via `/api proxy`.
If the proxy encounters a 404/502, it falls back to `http://localhost:5000/api`, with secondary fallback to direct Supabase client.

Cloud Deployment Mode (Vercel)

Automatically detected via `window.location.hostname` `≠ 'localhost'`. Bypasses Express proxy to communicate directly with Supabase via secure TLS, preventing Mixed Content and 404 failures.

5. WebRTC P2P Transfer & Signaling Protocol

Local high-throughput file transfers execute via WebRTC SCTP DataChannels following a deterministic signaling state machine over WebSockets:



6. REST API Endpoint Specifications

Method	Endpoint	Auth Header	Function
POST	/api/auth/register	None	Creates user in GoTrue + <code>public.users</code> .
POST	/api/auth/login	None	Issues JWT bearer token.
GET	/api/auth/me	Bearer <token>	Fetches profile, PIN lock state, question flag.
POST	/api/pin/create	Bearer <token>	Configures master PIN hash.
POST	/api/pin/verify	Bearer <token>	Validates PIN; returns <code>x-pin-token</code> .
POST	/api/pin/recover-with-question	Bearer <token>	Validates secret answer hash; resets PIN.
POST	/api/files/create-code	Bearer <token>	Saves Code Studio snippet to cloud vault.
GET	/api/udhar/summary	Bearer <token>	Calculates net receivable/payable balances.